Assignment Cover– be sure to keep a copy of all work submitted

To be completed by student – PLEASE PRINT CLEARLY

Name:
MUHAMMAD SYAMIRUL AZIM BIN AZMAN
ABDUL RAHMAN BIN ABDUL RAHIM
DANISH HAZIM BIN RAFIS SUZEIN

ID Number:
AM2512021407
AM2512021441
AM2512021357

Lecturer**: NURUL AZLIN LIYANA BINTI MOHAMAD REDZUAN**		Lab group / Tutorial group / Tutor (if applicable)
Course and Course Code: **SOFTWARE DEVELOPMENT PARADIGM / SWC 2623**		Submission Date: 30/9/2026
Assignment No. / Title: **Assignment**		Extension & Late submission: Allowed / Disallowed
Assignment type: ASSIGNMENT	% of Assignment Mark	Returning Date:

Penalties:

1. 10% of the original mark will be deducted for every one-week period after the submission date
2. No work will be accepted after two weeks of the deadline
3. If you were unable to submit the coursework on time due to extenuating circumstances you may be eligible for an extension
4. Extension will not exceed one week

Declaration: I/we the undersigned confirm that I/we have read and agree to abide by these regulations on plagiarism and cheating. I/we confirm that this piece of work is my/our own. I/we consent to appropriate storage of our work for checking to ensure that there is no plagiarism/ academic cheating. Signature:

azim danish

Full Name: MUHAMMAD SYAMIRUL AZIM BIN AZMAN
ABDUL RAHMAN BIN ABDUL RAHIM
DANISH HAZIM BIN RAFIS SUZEIN

PART A: FUNCTIONAL PROGRAMMING (HASKELL)

A1. Problem Description

The problem focuses on creating a Smart Learning Performance Analysis system using the Haskell programming paradigm. The system is designed to analyse learner performance based on assessment scores from three modules. Each learner is represented by a learner ID, name and a list of assessment scores.

The Smart Learning Performance Analysis system calculates the average score for each learner to provide a clear overview of their overall performance. The system also identifies high-achieving learners. A learner is considered a high achiever if their average score is 80 or above. In addition, the system determines the top-performing learner by comparing the average scores of all learners. When two or more learners have the same highest average score, the system applies a tie rule where the first learner appearing in the list is selected.

To ensure that the system works correctly, three test cases are included: a normal case, a boundary case where the average score is exactly 80, and a tie case where two learners have the same average score. These test cases allow the system's calculations and decision-making functions to be verified under different conditions.

A2. Design / Approach

2.1 Learner Data Representation

The program starts by defining a Learner data type using a tuple that contains three elements: a learner ID, a learner name and a list of assessment scores. Five learners are stored in a list called learners, with each learner having three assessment scores. The learner data is stored as an immutable list, which means the original data remains unchanged while the program processes and analyses the scores.

2.2 Average Score Calculation

The averageScore function is used to calculate the assessment average for each learner. It starts by retrieving the list of scores for a learner using pattern matching. Then, all the scores are added together using the sum function. After that, the total score is divided by the number of scores, which is determined using the length function. The result represents the learner's average score and helps to show their overall performance.

2.3 Identifying High Achievers

The program identifies high-achieving learners using the highAchievers function. This function uses filter to process the list of learners and checks whether each learner has an average score of 80 or above. Learners who meet this condition are included in the resulting list. This allows the program to automatically identify learners who meet the required performance level.

2.4 Finding the Top Performing Learner

The topLearner function determines the learner with the highest average score. It uses recursion to compare the current learner with the best learner from the remaining list. The function also applies a tie rule. When two or more learners have the same highest average score, the learner who appears first in the list is selected. In the provided dataset, Faris and Nadia both have an average score of 90.00. Therefore, Faris is selected because he appears first in the list.

2.5 Testing Approach

Three test cases are included to verify that the program produces the expected results under different conditions. The first test represents a normal case with an average score of 75.00. The second test represents a boundary case where the average score is exactly 80.00, confirming that the learner is classified as a high achiever. The third test represents a tie case where two learners have the same average score of 90.00. The result confirms that the first learner in the list is selected according to the defined tie rule.

2.6 Python Data Integration

The Haskell program also prepares learner data for integration with the Python component. After completing the learner performance analysis, the program produces a section labelled PYTHON_DATA. This section contains each learner's ID, name and calculated average score. The values are separated using the | symbol, allowing the Python program to read and process the data easily. This allows the results generated by Haskell to be used in the integrated learner dashboard.

A3. Selected Code Extract

3.1 Learner Data Structure

```
 8   type Learner = (String, String, [Double])
 9
10   learners :: [Learner]
11   learners =
12       [ ("L001", "Aina",  [85, 90, 88])
13       , ("L002", "Amir",  [75, 80, 85])
14       , ("L003", "Siti",  [80, 80, 80])
15       , ("L004", "Faris", [92, 88, 90])
16       , ("L005", "Nadia", [92, 88, 90])
17       ]
```

This code defines the structure of a learner using a tuple that contains the learner ID, learner name and a list of assessment scores. The learners list stores five learners, with each learner having three assessment scores. This dataset is used throughout the program to analyse learner performance.

3.2 Average Score Calculation

```
20   averageScore :: Learner -> Double
21   averageScore (_, _, scores) =
22       sum scores / fromIntegral (length scores)
```

The averageScore function calculates the average score of a learner. It uses pattern matching to retrieve the list of scores, calculates the total using the sum function, and divides the total by the number of scores determined using the length function.

3.3 High Achiever Function

```
33   highAchievers :: [Learner] -> [Learner]
34   highAchievers =
35       filter (\learner -> averageScore learner >= 80)
```

The highAchievers function identifies learners who have an average score of 80 or above. The filter function processes each learner in the list and keeps only those who meet the required average score condition.

3.4 Top Performing Learner Function

```haskell
topLearner :: [Learner] -> Learner
topLearner [] =
    error "Learner list cannot be empty."

topLearner [x] = x

topLearner (x:xs)
    | averageScore x >= averageScore best = x
    | otherwise = best
    where
        best = topLearner xs
```

The topLearner function uses recursion to determine the learner with the highest average score. It compares the current learner with the best learner from the remaining list. If two learners have the same highest average score, the >= condition ensures that the learner appearing first in the list is selected.

3.5 Python Data Integration

```haskell
    putStrLn "\nPYTHON_DATA"

    mapM_
        (\learner ->
            putStrLn (
                learnerID learner
                ++ "|"
                ++ learnerName learner
                ++ "|"
                ++ show (averageScore learner)
            )
        )
        learners
```

This code prepares learner data for integration with the Python component. The PYTHON_DATA label marks the beginning of the structured learner data. The mapM_ function processes each learner and outputs the learner ID, name and calculated average score. The | symbol separates each value, making the output easier for the Python program to read and process. This allows the results generated by Haskell to be used in the integrated learner dashboard.

A4. Test Cases

The program was tested using three different test cases to ensure that the learner performance analysis functions work correctly. The first test is a normal case that verifies the average score calculation under normal conditions. The second test is a boundary case that checks whether a learner with an average score of exactly 80 is classified as a high achiever. The third test is a tie case that verifies how the program handles two learners with the same highest average score.

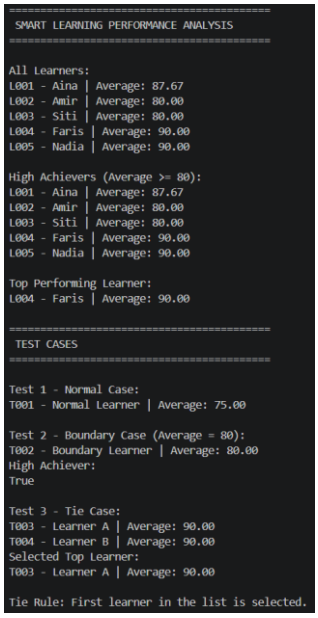
Test Case	Input / Condition	Expected Result	Actual Result	Status
TC01: Normal Case	Learner scores: [70, 75, 80]	Average score should be 75.00	Average displayed as 75.00	Pass
TC02: Boundary Case	Learner scores: [80, 80, 80]	Average should be 80.00 and learner should be classified as a high achiever	Average displayed as 80.00 and High Achiever: True	Pass
TC03: Tie Case	Learner A: [90, 90, 90] Learner B: [90, 90, 90]	Both learners have average 90.00. According to the tie rule, both learners have an average score of 90.00. According to the tie rule, Learner A should be selected because the first learner in the list is chosen.	Learner A was selected as the top learner	Pass

Test Result Explanation

The first test confirms that the program correctly calculates a normal learner's average score as 75.00. The second test checks the boundary condition where the average score is exactly 80.00 and confirms that the learner is classified as a high achiever. The third test verifies the tie-handling rule by using two learners with the same highest average score of 90.00. The result shows that Learner A is selected because the learner appearing first in the list is chosen according to the defined tie rule.

A5. Program Output and Screenshot

The screenshots below show the execution of the Haskell Smart Learning Performance Analysis program. The output displays the average score for each learner, identifies the high-achieving learners, determines the top-performing learner, and shows the results of the three test cases. The program also produces structured learner data for integration with the Python component.



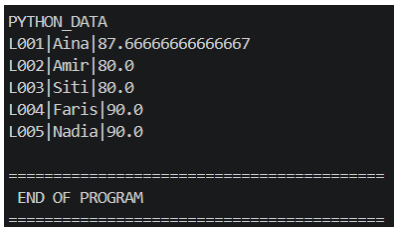


Figure 1: Output of the Haskell Learning Performance Analysis Program

Figure 2: Haskell Data Output for Python Integration

The program successfully calculates the average score for all five learners. Each learner has an average score of 80 or above, so all five learners are identified as high achievers. Faris is selected as the top-performing learner with an average score of 90.00. Although Nadia also has an average score of 90.00, Faris is selected because the tie rule prioritizes the learner who appears first in the list.

The test case section confirms that the program handles different conditions correctly. The normal test case produces an average score of 75.00. The boundary test confirms that an average score of exactly 80.00 is classified as a high achiever. The tie test verifies that when two learners have the same highest average score, the learner appearing first in the list is selected.

In addition, the program produces a PYTHON_DATA section for integration with the Python component. This section displays each learner's ID, name and calculated average score, with each value separated by the | symbol. The output confirms that the learner performance

results generated by Haskell are successfully prepared for use by the Python program in the integrated learner dashboard.

A6. Functional Programming Explanation

The Haskell program applies several functional programming principles to analyse learner performance. Firstly, pure functions are used throughout the program. For example, the averageScore function receives a learner as input and returns the calculated average without changing the original learner data or producing side effects. This ensures that the same input always produces the same result.

Immutability is also demonstrated through the learners dataset. The learner information and scores remain unchanged during the analysis. Instead of modifying the original list, new results are produced by applying functions to the learner data.

The program also uses pattern matching in functions such as averageScore, learnerID, and learnerName to extract specific information from the learner tuple. In addition, the highAchievers function uses the higher-order function filter with a lambda expression to process the learner list and select learners with an average score of 80 or above.

Finally, recursion is applied in the topLearner function to compare learners and determine the top-performing learner. These features demonstrate how Haskell uses functional techniques instead of traditional imperative loops and mutable variables.

A7. Conclusion

The Haskell Learning Performance Analysis program successfully analyses learner assessment scores using a functional programming approach. The program calculates the average score for each learner, identifies high-achieving learners with an average score of 80 or above, and determines the top-performing learner based on the highest average score. The defined tie rule also ensures that learners with the same highest average score are handled consistently by selecting the learner who appears first in the list.

The normal, boundary and tie test cases produced the expected results, showing that the implemented functions work correctly under different conditions. Overall, the program demonstrates the effective use of Haskell features such as pure functions, immutability, pattern matching, higher-order functions, list processing and recursion to solve the learner performance analysis problem.

Additionally, the program outputs structured learner data containing the learner ID, name and calculated average score for use by the Python component. This allows the results generated by Haskell to be used in the integrated learner dashboard.

PART B - LOGIC PROGRAMMING (PROLOG)

B1. Problem Description

The Smart Learning Pathway and Certification System uses Prolog logic programming. I see that the Smart Learning Pathway and Certification System is designed to give advice about which modules a learner should enrol in and to decide if a learner can earn a programme certification. The Smart Learning Pathway and Certification System keeps records of learners modules that are available modules that a learner has finished the prerequisites, for each module and the modules that the programme requires.

The Smart Learning Pathway and Certification System uses rules to figure out whether a learner can enrol in a particular module. A learner is considered eligible when the module exists the learner has not already completed it and all required prerequisites are satisfied. The Smart Learning Pathway and Certification System can also recommend modules that the learner's currently able to take. Additionally, the certification rule checks whether the learner has finished every module that the programme demands.

The Smart Learning Pathway and Certification System contains data for five learners and six modules. Different completion records are given to the learners so that we can see examples of both unsuccessful eligibility, recommendation and certification.

B2. Design and Approach

Prolog system is built with facts and rules. Facts give fixed information, about the learning environment. Rules use those facts to create fresh information.

The main predicates used are:

- learner/2 – stores learner names and their programme.
- module/1 – stores the available modules.
- completed/2 – records modules completed by each learner.
- prerequisite/2 – defines the prerequisite relationship between modules.
- required_module/1 – identifies modules required for certification.
- eligible/2 – determines whether a learner can enrol in a module.
- recommend/2 – recommends modules that a learner is eligible to take.
- certified/1 – determines whether a learner has completed all required modules.

The system has six modules: Programming Fundamentals, Database Systems, Data Structures, Web Development, Software Engineering and Artificial Intelligence. Programming

Fundamentals has no prerequisite. The remaining modules need modules to be finished first. This setup lets the system show how to check prerequisites and to infer logic.

B3. Selected Code Extracts

Eligibility Rule

The eligible/2 rule determines whether a learner is allowed to enrol in a target module.

```
eligible(Learner, Module) :-
    module(Module),
    \+ completed(Learner, Module),
        prerequisites_satisfied(Learner, Module).


recommend(Learner, Module) :-

     eligible(Learner, Module).



certified(Learner) :-

learner(Learner, _),

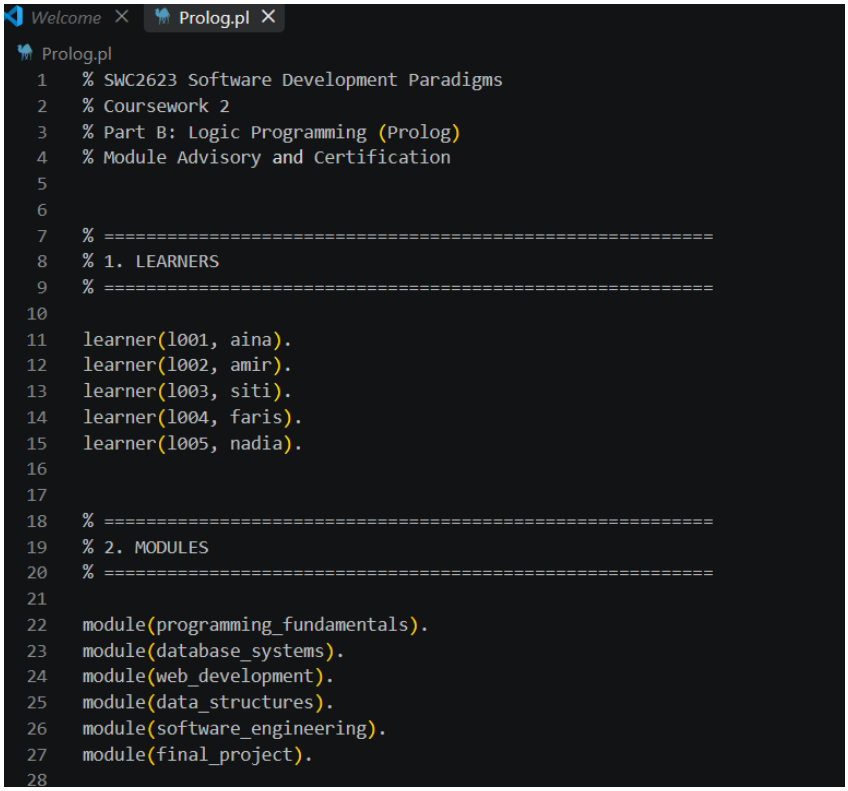
forall( required_module(Module),

completed(Learner, Module) ).
```

B4. Test Table

No	Input / Query	Expected Result	Actual Result	Status
1.	eligible(ali, data_structures).	true	true	PASS
2.	eligible(ali, artificial_intelligence).	false	false	PASS
3.	recommend(ali, Module).	data_structures, web_development	data_structures, web_development	PASS
4.	recommend(ahmad, Module).	database_system, data_structures, web_development	database_system, data_structures, web_development	PASS
5.	eligible(siti, software engineering).	true	true	PASS
6.	certified(faris).	true	true	PASS
7.	certified(ali).	false	false	PASS
8.	certified(nur).	false	false	PASS

B5. Query and Output



Welcome | Prolog.pl

Prolog.pl

```
% ============================================================
% 3. COMPLETED MODULES
% ============================================================

completed(l001, programming_fundamentals).
completed(l001, database_systems).

completed(l002, programming_fundamentals).

completed(l003, programming_fundamentals).
completed(l003, database_systems).

completed(l004, programming_fundamentals).
completed(l004, database_systems).
completed(l004, web_development).
completed(l004, data_structures).
completed(l004, software_engineering).
completed(l004, final_project).

completed(l005, programming_fundamentals).
completed(l005, web_development).


% ============================================================
% 4. PREREQUISITES
% ============================================================

prerequisite(database_systems, programming_fundamentals).
prerequisite(web_development, programming_fundamentals).
prerequisite(data_structures, programming_fundamentals).
prerequisite(software_engineering, database_systems).
prerequisite(software_engineering, data_structures).
prerequisite(final_project, software_engineering).
```

Welcome | Prolog.pl

Prolog.pl

```
% ============================================================
% 5. PROGRAMME REQUIRED MODULES
% ============================================================

required_module(programming_fundamentals).
required_module(database_systems).
required_module(web_development).
required_module(data_structures).
required_module(software_engineering).
required_module(final_project).


% ============================================================
% 6. ELIGIBILITY RULE
% ============================================================

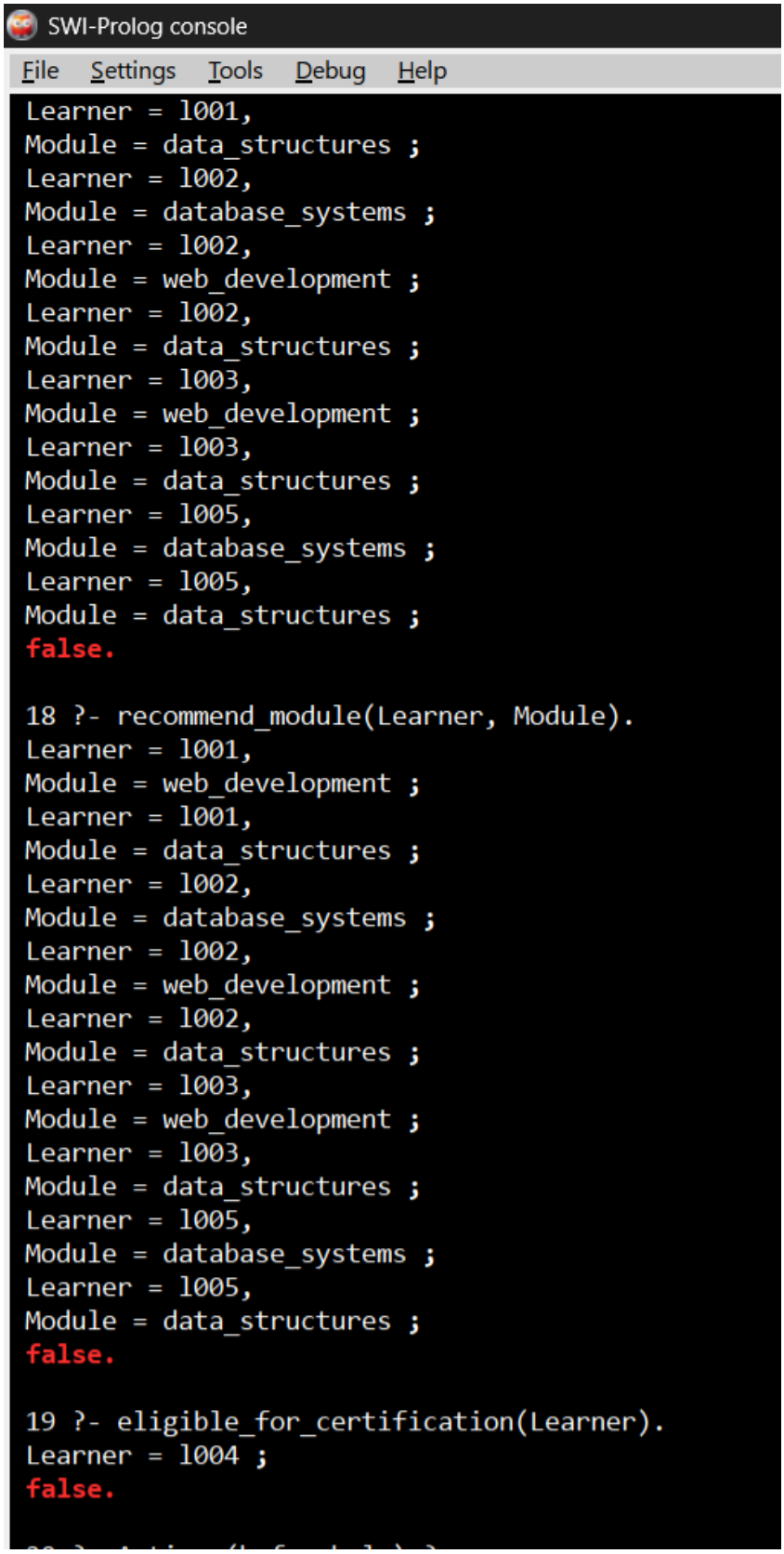
% A learner is eligible if:
% 1. The module exists
% 2. The learner has not completed it
% 3. All prerequisites have been completed

eligible(Learner, Module) :-
    learner(Learner, _),
    module(Module),
    \+ completed(Learner, Module),
    forall(
        prerequisite(Module, Required),
        completed(Learner, Required)
    ).
```


Welcome
Prolog.pl
Prolog.pl
96 % ==========
97 % 7. RECOMMENDATION RULE
98 % ==========
99
100 recommend_module(Learner, Module) :-
101 eligible(Learner, Module).
102
103
104 % ==========
105 % 8. CERTIFICATION RULE
106 % ==========
107
108 eligible_for_certification(Learner) :-
109 learner(Learner, _),
110 forall(
111 required_module(Module),
112 completed(Learner, Module)
113).


SWI-Prolog console
File Settings Tools Debug Help
Welcome to SWI-Prolog (threaded, 64 bits, version 10.0.2)
SWI-Prolog comes with ABSOLUTELY NO WARRANTY. This is free software.
Please run ?- license. for legal details.
For online help and background, visit https://www.swi-prolog.org
For built-in help, use ?- help(Topic). or ?- apropos(Word).
13 ?- learner(ID, Name).
ID = l001,
Name = aina ;
ID = l002,
Name = amir ;
ID = l003,
Name = siti ;
ID = l004,
Name = faris ;
ID = l005,
Name = nadia.
13 ?- module(Module).
Module = programming_fundamentals ;
Module = database_systems ;
Module = web_development ;
Module = data_structures ;
Module = software_engineering ;
Module = final_project.
14 ?- completed(Learner, Module).
Learner = l001,
Module = programming_fundamentals ;
Learner = l001,
Module = database_systems ;
Learner = l002,
Module = programming_fundamentals ;
Learner = l003,
Module = programming_fundamentals ;
Learner = l003,
Module = database_systems ;
Learner = l004,
Module = programming_fundamentals ;
Learner = l004,
Module = database_systems ;
Learner = l004,


SWI-Prolog console
File Settings Tools Debug Help
Learner = l004,
Module = data_structures ;
Learner = l004,
Module = software_engineering ;
Learner = l004,
Module = final_project ;
Learner = l005,
Module = programming_fundamentals ;
Learner = l005,
Module = web_development.
15 ?- prerequisite(Module, Required).
Module = database_systems,
Required = programming_fundamentals ;
Module = web_development,
Required = programming_fundamentals ;
Module = data_structures,
Required = programming_fundamentals ;
Module = software_engineering,
Required = database_systems ;
Module = software_engineering,
Required = data_structures ;
Module = final_project,
Required = software_engineering.
16 ?- required_module(Module).
Module = programming_fundamentals ;
Module = database_systems ;
Module = web_development ;
Module = data_structures ;
Module = software_engineering ;
Module = final_project.
17 ?- eligible(Learner, Module).
Learner = l001,
Module = web_development ;
Learner = l001,
Module = data_structures ;
Learner = l002,
Module = database_systems ;
Learner = l002,
Module = web_development ;
Learner = l002,


SWI-Prolog console
File Settings Tools Debug Help
Learner = l001,
Module = data_structures ;
Learner = l002,
Module = database_systems ;
Learner = l002,
Module = web_development ;
Learner = l002,
Module = data_structures ;
Learner = l003,
Module = web_development ;
Learner = l003,
Module = data_structures ;
Learner = l005,
Module = database_systems ;
Learner = l005,
Module = data_structures ;
false.
18 ?- recommend_module(Learner, Module).
Learner = l001,
Module = web_development ;
Learner = l001,
Module = data_structures ;
Learner = l002,
Module = database_systems ;
Learner = l002,
Module = web_development ;
Learner = l002,
Module = data_structures ;
Learner = l003,
Module = web_development ;
Learner = l003,
Module = data_structures ;
Learner = l005,
Module = database_systems ;
Learner = l005,
Module = data_structures ;
false.
19 ?- eligible_for_certification(Learner).
Learner = l004 ;
false.

B6. Explanation of Logic Programming

The Prolog knowledge base is an implementation of a Smart Learning Pathway and Certification System based on the logic programming paradigm. Facts are used to store basic information about learners, modules, completed modules, prerequisites and programme-required modules. For example, learner(ali, diploma_it) means Ali is a learner in the Diploma IT program, and completed(ali, database_systems) means a completed module.

Rules are used for deriving new knowledge from already known facts. The eligible/2 rule checks whether a learner is eligible to enroll in a module. It checks whether the module exists, whether the module has not been completed, and whether all prerequisites have been met. The recommend/2 rule suggests suitable modules using the eligibility rule. The certified/1 rule verifies that all required modules are completed.

Variables such as Learner , Module and RequiredModule allow the same rules to be used with different learners and modules . During query processing Prolog can use unification to match variables against facts. Prolog employs inference, in which it searches the knowledge base and rules to find out if a query is true or false. This allows queries to return logical results from the stored facts and rules.

B7. Conclusion

The Prolog implementation results in a successful rule based module advisory and certification system. The knowledge base contains information on learner progress, module prerequisites and program requirements. Logical rules are used to determine module eligibility, provide recommendations and check certification status. The test results show both successful and unsuccessful cases. Illustration of the ability of the logic programming paradigm to solve the learning pathway problem with facts, rules, variables, unification and inference.

PART C: MULTI-PARADIGM PROGRAMMING (PYTHON)

C1. Problem Description

Learner performance data can be difficult to manage and may be time-consuming and error-prone when done manually. It can also be difficult to filter and sort, create specialized groups, and organize the learner's information. These issues can be prevented with the Integrated Learner Dashboard. It was developed by combining multiple “programming paradigms such as functional programming, logic programming, and object-oriented programming using Python, Haskell, and Prolog.” Haskell is used to handle the learner's performance data that outputs the learner's name and its average score. Prolog contains facts and rules that expose the completed modules, module recommendations, and certification status. Python acts as the central integrating language, harvesting information from the other two through a defined interface and creating and populating Learner objects.

The integrated system keeps track of learner information, including: learner ID, name, modules completed and average score, and validates the integrity of all information via input validation. Python takes the Average Scores output from Haskell to categorise the level of the learner's success, find the high scorers and rank the learners based on their performance. The integrated information is output to the Integrated Learner Dashboard and a structured Pandas summary table. Test cases are included to test normal, boundary, low and invalid averages.

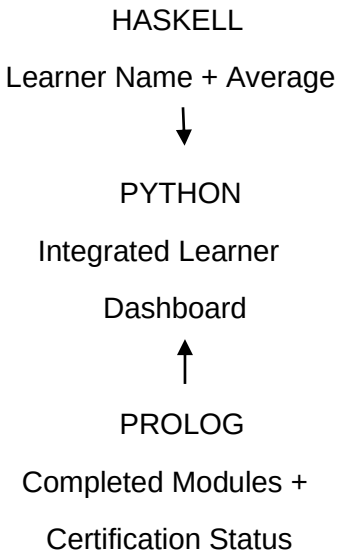
C2. Objectives

Overall Goal The objective of the project is to design a multi-paradigm learner information system based on Python, Haskell, and Prolog. The Learner information system stores learner information including learner id, name, the modules completed by the learner, and the average score by implementing Learner class in Python. The goal of the system is to employ Haskell to supply a list of learner names and average score, Prolog to supply facts and rules about a learner's completed modules, recommended modules, and certification, and Python as the main integration language which accesses and combines the information from Haskell and Prolog and represents it as Python objects called Learners. Another goal is to create an example of Object-Oriented Programming and functional-style programming in Python. Learner objects store information about individual learners. The python built-in functions map(), filter(), sorted(), lambda functions and list_comprehension are used to perform various operations on the information stored for Learners, such as retrieving average scores, finding good students, extracting just learner names, and ranking students based on their average score. The system will also use of the Pandas library to display integrated learner data in a summarised table. Validation on input and included test cases for normal, border, low and

invalid average inputs. Overall the project illustrates the ability to combine multiple programming paradigms in a single learner performance analysis system.

C3. System Design / Approach

C3.1 Integration Architecture



The core integration platform of the system is Python. Haskell handles the information about the learners and returns the name of the learner along with the average score. Prolog contains the logical knowledge about which modules have been completed, which modules are recommended and which modules lead to certification. Python runs these external programs and collects their output and recombines them into the Python data structures in the form of Learner objects.

C3.2 Haskell Integration

```python
def get_haskell_data():
    try:
        result = subprocess.run(
            ["runghc", "test.hs"],
            capture_output=True,
            text=True,
            check=True
        )

        output = result.stdout

        if "PYTHON_DATA" not in output:
            return ""

        return output.split("PYTHON_DATA", 1)[1]

    except Exception as error:
        return f"Haskell Error: {error}"
```

```python
def parse_haskell_data():
    raw_data = get_haskell_data()
    haskell_data = {}

    for line in raw_data.strip().splitlines():
        parts = line.split("|")

        if len(parts) == 3:
            learner_id = parts[0].strip()
            name = parts[1].strip()

            try:
                average = float(parts[2].strip())
            except ValueError:
                continue

            haskell_data[learner_id] = {
                "name": name,
                "average": average
            }

    return haskell_data
```

The gethaskelldata() function runs the Haskell program and collects the output, and the parsehaskelldata() function cleans up the data, ultimately generating the learner ID, name, and average score. This output is saved into a Python dictionary for use with the Integrated Learner Dashboard.

C3.3 Prolog Integration

```python
def get_prolog_data():
    try:
        goal = (
            "forall(learner(L,Name),"
            "(findall(M,completed(L,M),Modules),"
            "(eligible_for_certification(L) -> Status='Eligible' ; Status='Not Eligible'),"
            "write(L),write('|'),"
            "write(Name),write('|'),"
            "write(Modules),write('|'),"
            "writeln(Status))),"
            "halt"
        )

        result = subprocess.run(
            [
                "swipl",
                "-q",
                "-s", "test.pl",
                "-g", goal
            ],
            capture_output=True,
            text=True,
            check=True
        )

        return result.stdout

    except Exception as error:
        return f"Prolog Error: {error}"
```

```python
def parse_prolog_data():
    raw_data = get_prolog_data()
    prolog_data = {}

    for line in raw_data.strip().splitlines():
        parts = line.split("|")

        if len(parts) == 4:
            learner_id = parts[0].upper()
            name = parts[1].capitalize()
            modules_text = parts[2].strip("[]")

            if modules_text:
                modules = [
                    module.strip().replace("_", " ").title()
                    for module in modules_text.split(",")
                ]
            else:
                modules = []

            status = parts[3].strip()

            prolog_data[learner_id] = {
                "name": name,
                "completed_modules": modules,
                "certification": status
            }

    return prolog_data
```

The getprologdata() function runs the Prolog program to retrieve the learner info (such as what modules they have completed or whether or not they are certified). The parseprologdata() function formats the Prolog output into a structured Python dictionary that can be added to the Integrated Learner Dashboard.

C3.4 Learner Class

The Learner class is the Object-Oriented representation of the integrated Python program. Each Learner object would have 4 attributes: learner_id, name, completed_modules, and average. The learner name and average were fetched from Haskell, whereas the completed module data was fetched from Prolog. Python would then join these values into the single Learner object.

The constructor also validates input. Learner ID and name may not be empty; averagescore must be between 0 and 100. The average_score() method yields the average value supplied by the Haskell component. The display() method displays the learner ID, name, average score and performance classification.

C3.5 Functional Programming Techniques

For the collection of Learner objects, the Python program uses the following functional-style programming features: map(), filter(), sorted(), lambda, list comprehensions - elegant ways to perform operations on list/ collection data.

Map() is used together with a lambda expression to obtain the average score of each Learner object. The expression lambda learner: learner.average_score() is run for each learner and results in a list of average scores of the learners without changing the original object.

Another use of the filter() function is to be combined with a lambda expression for making a filter using some condition. In the example, the condition we want lambda learner: learner.average_score() >= 80 will only select the learners who scored 80 or higher. The selected Learner objects are assigned to the list highachievers.

4.9 How do I get information out of Learner objects? You will have a list of Learner objects called learners and you want to get all the learner names. List comprehension is needed here. [learner.name for learner in learners] generates a list of learner names.

Lastly, a sorted() method is used with a lambda expression to sort the learners based on their average marks. The lambda learner: learner.average_score() expression is used as the key in the sorted() method and reverse = True is used to arrange the learners in reverse order i.e. from most scores to lowest scores. The sorted list is stored in ranked_learners.

C3.6 Performance Classification

The program uses a helper function called classify_performance() to decide how well a student did. The helper function takes one number, the score and checks it against a list of limits. If the average is 80 or higher classify_performance() returns the word "Excellent”. If the average falls between 70 and 80 classify_performance() returns "Good”. If the average is between 60 and 70 classify_performance() returns "Satisfactory”. When the average is below 60, classify_performance() returns "Needs Improvement”.

This classification logic is sewn into the Learner class so that the result can be shown automatically. Inside the Learner class the display() method calls self.average_score(). Hands that number straight to classify_performance(), inside an f-string. Because classify_performance() does all the deciding the program can print each student’s ID, name, average and the qualitative label without repeating any logic.

C3.7 Pandas Integration

Presented in a tabular format for easy interpretation, the Pandas library is used to display the integrated learner data in a summary table. The code initially generates summary_data that holds the information from each of the Learner object, namely the learner id, learner name, total modules completed, average score, learner performance, and certification status.

The completed modules are calculated as len(learner.completed_modules). And the average

score is retrieved with learner.average_score() and rounded to 2 decimal places. Finally, for each learner the classifies whether they is Excellent, Good, Satisfactory or Needs Improvement using the classify_performance() function based on average score.

The certification status is selected from the Prolog data via the learner ID. That proves the link between Prolog and Python: the certification result obtained from the Prolog component is set along the learner data in the Pandas output.

A simplified learnersummary may be generated by creating a list of learner info dictionaries using summarydata and then converting it into a Pandas dataframe with pd. DataFrame(summary_data). Df.to_string(index=False) provides a human readable render of the dataframe learner_summary.

C4. Selected Code Extracts

C4.1 Learner Class

```python
class Learner:
    def __init__(self, learner_id, name, completed_modules, average):

        if not learner_id or not name:
            raise ValueError("Learner ID and name cannot be empty.")

        if average < 0 or average > 100:
            raise ValueError("Average must be between 0 and 100.")

        self.learner_id = learner_id
        self.name = name
        self.completed_modules = completed_modules
        self.average = average
```

The Learner class is used to describe each Learner in the integrated system. The init( ) constructor creates a new Learner object with four attributes: learner_id, name, completed_modules, and average.

The constructor includes input validation that validates the input for learner id and name that it is not an empty string and the input for the score is valid, between 0 and 100. ValueError is raised for invalid data values.

Once validation is done, the values are stored as instance attributes with self.learnerid, self.name, self.completedmodules, and self.average. In the integrated system, the learner name and average are taken from Haskell and completed modules are taken from Prolog. Python synthesizes this information into a structured Learner object.

C4.2 Average Score Retrieval

```
def average_score(self):
    return self.average
```

You can see the use of the average_score() method and the average score value stored in the Learner object. As the average is calculated in the Haskell part, no need to re-compute it in Python. The average returned value will be used for performance classification, filtering, ranking and Pandas summary.

```
Learner(
    "L001",
    haskell_data["L001"]["name"],
    prolog_data["L001"]["completed_modules"],
    haskell_data["L001"]["average"]
),
```

Creating an object When the object 'learner' is created the values “name and average score” are being taken from haskelldata and the value 'Completed modules' are taken from prologdata. This show how python combines data from haskell and prolog into one object 'learner'.

C4.3 Functional Operation 1 – Map

```
# MAP + LAMBDA
averages = list(
    map(lambda learner: learner.average_score(), learners)
)
```

The line uses map() together with a lambda expression (lambda learner: learner.average_score()) in order to transform a list of Learner instances into a pure list of numbers. Map iterates over each item in learners, and then applies the average_score() function to each object. Finally, the whole thing is converted back into a list.

The map() function is perfectly suited because it does a clean 1-to-1 transform from Learner instances to number values without actually doing anything to modify the data in the Learner instances.

As with the use of lambda, map() in this form facilitates declarative programming with no explicit loops, indexing variables, or empty lists waiting for populated values; it results in much cleaner and more legible code.

C4.4 Functional Operation 2 – Filter

```
# FILTER + LAMBDA
high_achievers = list(
    filter(lambda learner: learner.average_score() >= 80, learners)
)
```

The code employs a high-order function called filter () with an anonymous lambda function, that is: lambda learner: learner.average_score () >= 80) to select only the students that are good performers. Evaluating every Learners' average score with respect to 80 gives us a bool object which is then filtered so as to give back only those learner objects that satisfy the condition with a filter () method of a iterator, which later gets converted to a list in Python called high achivers with an outer list () function call. The filter () method is most appropriate in this case, due to the simple and declarative approach of selection based on logical expressions as compared to extensive loops with clear if statement and explicit. Append () method. It allows only those records to be included into the iteration that fulfill the conditional evaluation whereas leave other learner records unaffected.

C4.5 Functional Operation 3 - List Comprehension

```
# LIST COMPREHENSION
learner_names = [
    learner.name for learner in learners
]
```

This application of the program employs list comprehension to extract the learner ' s name from the learners list. As a result of each Iteration of Learner ' s Object, learner.name will be called so each learner name will then be added to the newly Created list learner_names.

C4.6 Functional Operation 4 – Sorted and Lambda

```
ranked_learners = sorted(
    learners,
    key=lambda learner: learner.average_score(),
    reverse=True
)
```

The sorted() function returns a list of learners sorted based on the average_score() function. The lambda learner: learner.average_score() evaluates each learner returned by getlearners() to get the average score for each learner. Sorted() sorts the learners in reverse order for the learner ranking. This list of sorted learners is stored in ranked_learners.

C4.7 Performance Classification

```
def classify_performance(average):
    if average >= 80:
        return "Excellent"
    elif average >= 70:
        return "Good"
    elif average >= 60:
        return "Satisfactory"
    else:
        return "Needs Improvement"
```

The function to classify the performance is represented in a conditional aspect to link the numerical value (average grade) with its corresponding categorical value (academic standing); this is accomplished in the classify_performance function which consists in the use of a simple if-elif-else block where it evaluates on which bracket the average will be based to return a specific category: “If the average is 80 or above, the function returns Excellent. If it is 70 or above, it returns Good. If it is 60 or above, it returns Satisfactory. Otherwise, it returns Needs Improvement.” For whatever grade will be under 60. So this is a self contained unit to avoids duplicate code as much as possible so every time one wanted to get the categories of each student regardless it was going to be displayed, or made into functional maps or aggregated using Pandas' facilities.

C4.8 Pandas Implementation

```
import pandas as pd
```

The Pandas library is also incorporated in to the application import pandas as pd. The purpose of using the Pandas library into application was to make the intricate object oriented nature of learning data structured within a presentation ready summary report. Using Pandas Data Frame a single learning object's (e.g.

By student ID, name, number of modules studied, average score obtained, current proficiency level, etc.) discrete characteristics are bound together to create a meaningful two dimensional table.

Pandas was incorporated with the application on the basis of its proficient data structuring mechanisms and useful methods regarding string formatting. It eliminated the need of careful whitespace configuration or tedious string concatenation using the method df.to_string(index=False) which perfectly formats the strings in human readable terminal tables. Moreover it renders easier scalability to long term applications to extract dashboard information to various kinds of external files like csv and excel and execute complex aggregations on the data.

```
df = pd.DataFrame(summary_data)

print("\nPANDAS LEARNER SUMMARY")
print(df.to_string(index=False))
```

This section code calls for and runs final data rendering stage by first initializing and printing a pandas DataFrame to the console. First step df = pd. DataFrame(summarydata), translates the list of dictionaries and consolidations of (summarydata) to the structured, 2 dimensional DataFrame object df which directly gets converted to columns based on its dictionary keys, then print("\nPANDAS LEARNER SUMMARY") prints a formatted header followed by the new line character; for clarity between separated sections and last, print(df.to_string(index=False)) which formats df into one long string based table in string, as index argument is set to FALSE in order to eliminate the default Pandas Indexing that it provides.

```
  "Certification": prolog_data[
      learner.learner_id
  ]["certification"]
```

There is a Certification column in the Pandas summary which we fill with the Certification information of the learners from the Prolog database. This is how the Python, Prolog and Pandas weave work together; the Prolog certification result is worked into the learner information and returned to Python, all in a dataframe.

C5. Test Cases

Test	Description	Input	Expected Result	Status
TC01	Normal Input	Average = 75.0	Good	PASS
TC02	Boundary Input	Average = 80.0	Excellent	PASS
TC03	Low Performance	Average = 50.0	Needs Improvement	PASS
TC04	Invalid Average	Average = 105.0	ValueError	PASS

Test Cases Test case are created to test the working of the class Learner and also to test the classification of performance. TC01 is designed to test the normal average value, TC02 is designed to test the boundary value 80.0, TC03 is designed to test the low performance value

and TC04 is created for testing that an average value out of range 0-100 should raise Value Error.

C6. Program Output / Screenshots

C6.1 Haskell Program Output

```
========================================
 SMART LEARNING PERFORMANCE ANALYSIS
========================================

All Learners:
L001 - Aina | Average: 87.67
L002 - Amir | Average: 80.00
L003 - Siti | Average: 80.00
L004 - Faris | Average: 90.00
L005 - Nadia | Average: 90.00

High Achievers (Average >= 80):
L001 - Aina | Average: 87.67
L002 - Amir | Average: 80.00
L003 - Siti | Average: 80.00
L004 - Faris | Average: 90.00
L005 - Nadia | Average: 90.00

Top Performing Learner:
L004 - Faris | Average: 90.00
```

Figure C6.1: Output from the Haskell Program

The output indicates all learners with their average scores, high achievers (learner's with an average score of 80 or more), and top performing learner. In this output Faris has been identified as the top performing learner with an average score of 90.00.

C6.2 Haskell Data Received by Python

```
HASKELL DATA RECEIVED BY PYTHON
L001 Aina | Average: 87.67

L002 Amir | Average: 80.00

L003 Siti | Average: 80.00

L004 Faris | Average: 90.00

L005 Nadia | Average: 90.00
```

Figure C6.2: Presents the Haskell data that has been received by and processed within Python.

This Python process takes the output generated by the Haskell code and extracts the learner ID, learner name, and learner score for each learner from the Haskell component output and then stores this information in a Python dictionary for display in the Integrated Learner Dashboard.

C6.3 Prolog Program Output

```
================================================
PROLOG - MODULE ADVISORY AND CERTIFICATION
================================================

LEARNERS:
l001 - aina
l002 - amir
l003 - siti
l004 - faris
l005 - nadia

ELIGIBLE MODULES FOR L001:
web_development
data_structures

RECOMMENDED MODULES FOR L001:
web_development
data_structures

CERTIFICATION STATUS:
l001 - aina : Not Eligible
l002 - amir : Not Eligible
l003 - siti : Not Eligible
l004 - faris : Eligible
l005 - nadia : Not Eligible
```

Figure C6.3 gives the output of the Prolog program. The output shows those learners who are registered, those modules that learner L001 is eligible for, those modules that learner L001 is recommended for, and whether a particular learner has been certified. The program applies facts and rules to arrive at this output.

C6.4 Prolog Data Received by Python

```
PROLOG DATA RECEIVED BY PYTHON
L001 Aina | Modules: 2 | Certification: Not Eligible
L002 Amir | Modules: 1 | Certification: Not Eligible
L003 Siti | Modules: 2 | Certification: Not Eligible
L004 Faris | Modules: 6 | Certification: Eligible
L005 Nadia | Modules: 2 | Certification: Not Eligible
```

Figure C6.4: The outputs of Prolog consumed by Python is presented here.

This output contains the learner's ID, name, number of modules completed, and certification information fetched from Prolog. Python collects the outputs from Prolog and saves the data in the dictionary from where the data is used for the learning dashboard.

C6.5 Integrated Learner Dashboard

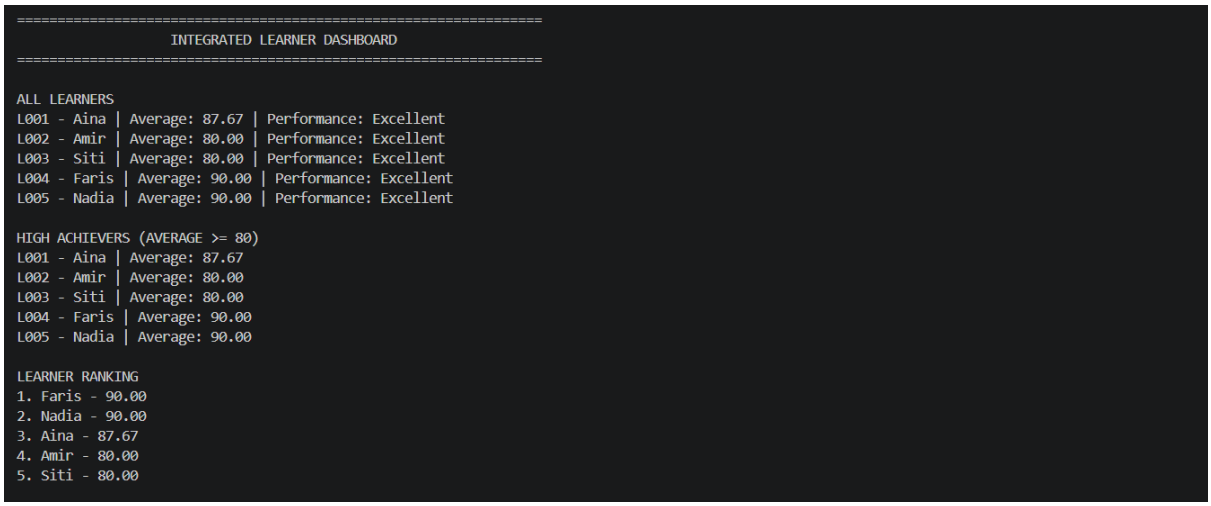


Figure C6.5: Displays Integrated Learner Dashboard produced by Python.

The learner ID, Name, average score, and Learner worked performance classification are listed with their high gainers having an average score of more than or equal to 80 are identified and their scores are ordered from the maximum to minimum. The output shows how Python generates the received learner data from the integrated components in the structured format.

C6.6 High Achievers



Figure C6.6 The high achievers output using the Python program

The filter() function with a lambda expression is used to extract all learners who have an average score of 80 and above. We can see from the output that Aina, Amir, Siti, Faris and Nadia are the high achievers because their average score is 80 or more.

C6.7 Learner Ranking

```
LEARNER RANKING
1. Faris - 90.00
2. Nadia - 90.00
3. Aina - 87.67
4. Amir - 80.00
5. Siti - 80.00
```

Figure C6.7 shows the ranking of learners by using the sorted() function with lambda expression that sorts the learners in decreasing order according to average score.

According to the results, Faris and Nadia obtained the maximum average score of 90.00, followed by Aina with 87.67, while Amir and Siti obtained the same minimum average score of 80.00.

C6.8 Pandas Integrated Summary

```
======================================================================
                        PANDAS LEARNER SUMMARY
======================================================================
Learner ID  Name  Completed Modules  Average Performance Certification
      L001  Aina                  2    87.67   Excellent  Not Eligible
      L002  Amir                  1    80.00   Excellent  Not Eligible
      L003  Siti                  2    80.00   Excellent  Not Eligible
      L004 Faris                  6    90.00   Excellent      Eligible
      L005 Nadia                  2    90.00   Excellent  Not Eligible
```

Figure C6.8. Learner information stored using a Pandas Data Frame.

The information on the learner ID, name, modules completed, mean score, classification of performance, and certification are presented in a tabular table.

C6.9 Test Cases Output

```
============================================================
                        TEST CASES
============================================================

Test 1 - Normal Input
T001 - Normal Learner | Average: 75.00 | Performance: Good

Test 2 - Boundary Input (Average = 80)
T002 - Boundary Learner | Average: 80.00 | Performance: Excellent

Test 3 - Low Performance
T003 - Low Performance Learner | Average: 50.00 | Performance: Needs Improvement

Test 4 - Invalid Average
Input Error: Average must be between 0 and 100.
```

Figure C6.9 Four test cases that can be used to test the program are illustrated in the figure.

The program performs correctly in classifying the normal, boundary and low performance test inputs, but the invalid average test verifies that values other than between 0 and 100 are correctly identified as invalid.

C7. Explanation of Multi-Paradigm Programming

Program paradigm: Demonstrates combination of multi paradigm programming styles. The Integrated Learner Dashboard brings together the following programming paradigms of different languages; Functional Programming, Logic Programming and Object-oriented Programming. For example, Haskell is used to implement the functional programming part of the dashboard to manipulate the information about learner performance, learner names, and average scores and is retrieved using Python.

Prolog is the logic programming part of the system. It encodes the facts and rules for storing learner module data, finding the qualifying and recommended modules, and checking certification status. Python interacts with the Prolog program and encodes the facts retrieved as structured data that can then be assimilated with the learner records.

Python is the primary integration language, which utilises Object-Oriented Programming and Functional style programming. The Learner object stores learner data: identifier, name, list of completed modules, and average score. Collection of Learner objects is manipulated using functional style operations like map(), filter(), sorted(), lambda expressions and list comprehension. For instance, high performers are selected using filter() and sorted() outputs the ranking of learners according to their averages.

Using Haskell, Prolog, and Python together within the same system shows how different programming styles can co-exist in the same system. The Haskell code does the processing, the Prolog code does the rule-based reasoning, and the Python code combines the results, displaying everything in the Integrated Learner Dashboard and Pandas summary.

C8. Conclusion

The Integrated Learner Dashboard has been able to incorporate both Haskell, Prolog and Python together to show multi-paradigm programming. Learner performance data in Haskell, Prolog module and certification data with logical rules, integrated data in Python via OOP and in functional style and a summary table on the learner information in Pandas all working together. Entirely different programming paradigms working harmoniously to show off one application.

References Haskell

- Marlow, S. (Ed.). (2010). *Haskell 2010 language report*. Haskell.org. https://www.haskell.org/onlinereport/haskell2010/
- Hudak, P., Peterson, J., & Fasel, J. (2000). *A gentle introduction to Haskell: Functions*. Haskell.org. A Gentle Introduction to Haskell: Functions

References Prolog

- GeeksforGeeks. (2026, January 25). Prolog | An introduction. https://www.geeksforgeeks.org/artificial-intelligence/prolog-an-introduction/
- TutorialsPoint. (n.d.). Prolog - basics. https://www.tutorialspoint.com/prolog/prolog_basics.htm

References Python

- Python Software Foundation. (2024). *Python documentation: Functional programming HOWTO*. https://docs.python.org/3/howto/functional.html
- The Pandas Development Team. (2024). *pandas-dev/pandas: Pandas (v2.2.0)*. Zenodo. https://doi.org/10.5281/zenodo.3509134
- Real Python. (2022). *Object-Oriented Programming (OOP) in Python 3*. https://realpython.com/python3-object-oriented-programming/
- Lutz, M. (2013). *Learning Python: Powerful Object-Oriented Programming* (5th ed.). O'Reilly Media.

ASSIGNMENT
SWC2623

STUDENT NAME _____
STUDENT NAME _____
STUDENT NAME _____

PART A: Functional Programming (Haskell)					
CRITERIA	**Excellent (8–10 marks)**	**Good (5–7 marks)**	**Satisfactory (3–4 marks)**	**Poor (0–2 marks)**	**MARKS**
Program correctness	All required functions implemented correctly; program runs flawlessly; accurate results for averages, distinctions	Minor logic or syntax issues; outputs mostly correct.	Major issues in logic or missing functionality; partial results.	Program fails to run or missing critical functions.	
Use of functional programming principles	Clear use of recursion, higher-order functions, and immutability. Demonstrates understanding of pure	Uses some functional features appropriately.	Limited use of functional concepts; mostly imperative.	No evidence of functional programming principles.	
Explanation (150 words)	Clearly explains how functional concepts were applied with relevant examples.	Explanation mostly clear but lacking detail or examples.	Minimal or unclear explanation.	Missing or irrelevant explanation.	
Part B: Logic Programming (Prolog)					
CRITERIA	**Excellent (8–10 marks)**	**Good (5–7 marks)**	**Satisfactory (3–4 marks)**	**Poor (0–2 marks)**	**MARKS**
Program correctness	All facts and rules implemented correctly; outputs for eligibility, recommendations, and graduation are accurate.	Mostly correct logic; minor issues with rule definitions or outputs.	Partial or inconsistent results; missing facts or rules.	Program does not run or rules are incorrect.	
Use of logic programming principles	Demonstrates excellent understanding of declarative problem-solving using facts and rules.	Appropriate use of logic principles with some minor flaws.	Limited or mechanical use of Prolog features.	No understanding of logic programming shown.	
Explanation (150 words)	Clear, detailed explanation linking logic programming concepts to implementation.	Mostly clear but lacks technical depth.	Minimal or vague explanation.	Missing or irrelevant explanation.	
Part C: Multi-Paradigm Programming (Python)					
CRITERIA	**Excellent (8–10 marks)**	**Good (5–7 marks)**	**Satisfactory (3–4 marks)**	**Poor (0–2 marks)**	**MARKS**
Program functionality	Fully implements class structure, functional operations, performance prediction, and library integration; outputs correct and relevant.	Minor bugs or incomplete features but demonstrates overall understanding.	Partial implementation; some errors or missing functionalities.	Does not meet requirements; fails to execute.	
Use of multiple paradigms	Demonstrates clear integration of OOP and functional paradigms (e.g., lambda, map/filter/reduce with class methods).	Reasonable integration with some imbalance between paradigms.	Limited use of multiple paradigms.	Only one paradigm used or no clear paradigm integration.	
Explanation (150–200 words)	Detailed and accurate explanation of how Python supports multiple paradigms, with examples.	Good explanation but lacks depth or examples.	Minimal or superficial explanation.	Missing or irrelevant explanation.	
Overall Report Quality					
CRITERIA	**Excellent (8–10 marks)**	**Good (5–7 marks)**	**Satisfactory (3–4 marks)**	**Poor (0–2 marks)**	**MARKS**
Report structure & submission	Report is well-organized, complete, and neatly formatted; includes group details, source code links, and screenshots.	Minor formatting or missing elements.	Incomplete report or missing one major section.	Disorganized or missing major deliverables.	